

These can also be deeply integrated with the build process such that the triggering of these reviews can be scheduled and performed in line with the build schedule, without manual intervention.

Selection criteria

After understanding the need for a code review tool, the next question we need to address is how to select this tool and what is the criteria we need to follow to assess the various tools available in the market. Is open source an option?

Some of the following features need to be assessed prior to deciding on the right open source tool for code reviews.

- **Coding standards and best practices**
The tool should alert the team whenever there is standards violation, increased complexity, etc. By enforcing these quality checkpoints, there will be better readability and maintenance of the code. Though there are many things possible here, starting out with simple features and extending them incrementally is the recommended approach.
- **DevOps integration**
The tool needs to have a rich API to interface with any other tools or technologies for effective automation and integration. It should work equally well with build integration tools like Maven and continuous integration engines like Jenkins. Also, there should be a provision to alert the success or failure of the build, with all the details required for possible troubleshooting.
- **Programming language support**
The tool should support the wide variety of programming languages in use today like C, C++, Java, JavaScript, etc. There should also be a provision to plug in or configure this tool with the IDE in use, like Eclipse, JDeveloper, etc.
- **Bug detection**
The bugs can be anything specific to typical coding standards and best practices, and sometimes even go a little deeper into security vulnerabilities. There should be a provision to enable project- or application-specific configurations and settings on a need basis. With all the digital options evolving at a rapid pace, security has become one of the important areas that need close monitoring. The tool should detect and arrest some of these security vulnerabilities early in the game. For example, the tool should never allow the hard coding of sensitive information like the Aadhaar number, IP address, etc, as part of the code.
- **Centralised quality**
We are slowly moving towards an enterprise kind of setup for code reviews, where the rules configured from individual projects can be shared and reused across other projects to ensure overall coverage. Also, the best practices evolved over a period of time can be shared across the teams to benefit the entire organisation. The tool should provide enough options to dashboard the

overall health, both at the organisation level (covering all projects) and at the individual project level. Also, the history gathered over a period of time should help in analysing the trends and patterns to define the path for continuous improvement.

Open source tools and options

Though there are many open source code review tools, to keep it simple, let's pick a few of them like PMD, FindBugs, CheckStyle and SonarQube for analysis.

Most of these tools generally support Java, and some of them extend this to other programming languages too. They typically have a plugin option to integrate with the development and deployment tools. The rules-driven approach and configurable nature makes them more saleable, comparatively. Features like CPD (copy paste detector) help in identifying and applying fixes across the code, at one shot.

1. **FindBugs** is a Java specific tool with little or no support for other programming languages. To keep it simple and easy for developers, it has an Eclipse IDE plugin for seamless detection and identification of issues. The recent addition of the cloud storage feature allows distributed access of bugs across the teams.
2. **PMD** has support for more programming languages than just Java, like PL/SQL, XML, etc. It has the Eclipse IDE plugin and integrates well with Maven for Build creation. Also, CPD is an inherent and important feature, which can help in identifying and fixing the duplicate code consistently across the board.
3. **Checkstyle** is another Java specific tool that works well with Eclipse. Its easy integration with Maven and Jenkins makes it more attractive. The highlight of this tool is its flexibility in adapting to the various coding standards prevailing in the industry.
4. **SonarQube** is an open source platform that helps in automating the end-to-end code review process. The differentiator of this tool is that it supports many programming languages along with Java. Like other tools, it also has plugins to work with the Eclipse IDE seamlessly, and its easy integration with Maven and Jenkins makes it a little more attractive. The highlight of this tool is its ability to work with multiple projects in parallel. Its rich dashboard feature makes it easy for the leadership team to get access to all the possible insights. SonarQube can be accessed online as a service and has the provision to configure some of the security-specific rules as well.

Source code can be fed as input for a tool, which is analysed to get the output as feedback. For the given input, the tool scans the entire code for defined rules and configurations, and outputs the metrics in a feature-rich dashboard for user consumption. It can also provide good insights on how the code is faring over a period of time, even comparing it to previous runs. This is to make sure that the